

Chapter One

Introduction to Programming

What is a programming?

- Computer
 - an electronic device that accepts data,
 - performs computations, and makes logical decisions according to instructions that have been given to it
 - then produces meaningful information in a form that is useful to the user
- Computer programs
 - Sets of instructions that control a computer's processing of data
 - the instructions that tells the computer what to do
- Computer programming
 - is the process of writing, testing, debugging / troubleshooting, and maintaining the source code of computer programs

Cont..

- Computer programs (also know as **source code**) is often written by professionals known as **Computer Programmers**
- A computer program usually consists of two elements:
 - Data – characteristics
 - Code – action
- Source code is written in one of programming languages
-

Many Aspects of Programming

- Programming is **problem solving**
 - always trying to make computer do something useful — i.e., finding an optimal travel route
- Programming is **controlling**
 - computer does exactly what you tell it to
- Programming is **teaching**
 - computer can only “learn” to do new things if you tell it how
- Programming is **creative**
 - must find a good solution out of many possibilities
- Programming is **modeling**
 - describe salient (relevant) properties and behaviors of a system of components (objects)
- Programming is **abstraction**
 - identify important features without getting  lost in detail

What's a Program?

- Model of complex system
 - *model*: simplified representation of salient features of something, either tangible or abstract
 - *system*: collection of components that work closely together
- Sequences of instructions expressed in specific programming language:
 - *syntax*: grammatical rules for forming instructions
 - *semantics*: meaning/interpretation of instructions
- Instructions written (programmed/coded) by programmer
 - coded in a specific programming language
 - as a result, programs cannot be ambiguous
 - all instructions together are called *source code*
 - Executed by computer by carrying out individual instructions



What is programming language

- an artificial language that can be used to control the behavior of a computer
- defined by
 - Syntactic - describes the possible combinations of symbols that form a syntactically correct program
 - Semantic - The meaning given to a combination of symbols
- Every programming languages have syntax and semantics
- computers do exactly what they are told to do
- Available programming languages come in a variety of forms and types
- Programming languages can be divided in to two major categories:
 - low-level
 - high-level languages

Programming Languages

- Machine language
 - machine is short for computing machine (i.e., computer)
 - computer's native language, sequence of zeroes and ones (binary)
 - different computers understand different sequences
 - hard for humans to understand: 01010001...
- Assembly language
 - mnemonics for machine language
 - *low level*: each instruction is minimal
 - still hard for humans to understand:
`ADD d0 ,d2`
- High-level languages
 - FORTRAN, Pascal, BASIC, C, C++, Java, etc.
 - *high level*: each instruction composed of many low-level instructions
 - closer to English easier to read and understand:
`hypotenuse = Math.sqrt(opposite *
opposite + adjacent * adjacent);` 

Interpreting vs. Compiling Programs

- Each type of computer only “understands” its own machine language (zeroes and ones)
- Thus we must translate from High-level language to machine language
 - a team of experts programs a translator, called a “compiler” which translates entirety of a High-level language program to an *executable file* in computer’s native machine language.
- Running :
 - *compilation: Your program → executable*
 - *execution: run executable*
 - machine executes your program by “running” each machine language instruction in the executable file



Interpreting vs. Compiling Programs

- An alternative to *compiling* your program is to *interpret* your program
 - each line of your program is translated into machine language and immediately executed
- Like translating between natural languages
 - **Compiler:** human translator translates book in its entirety and then translated book is printed and read
 - **Interpreter:** human interpreter translates each spoken statement in sequence as speaker is speaking



Programming paradigm

- A computer program usually consists of two elements:
 - Data – characteristics
 - Code – action

Some program development approaches conceptually organize the program around the code:

- Around what is happening
- known as Process oriented approach/ method, which characterizes the program as serious linear steps.
- Code acting on data

Other programming languages conceptually organize the program around data:

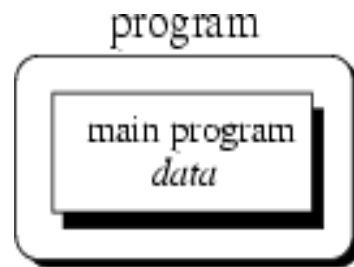
- Who is being affected
- designed to manage increasing complexity.
- Ask not what your code does to your data structure, but what your data structure can do for you

Programming paradigm...

- A **programming paradigm** is a fundamental style of programming
 - **Unstructured Programming**
 - **Procedural programming .**
 - **Structured Programming**
 - **Object Oriented Programming**

Unstructured Programming

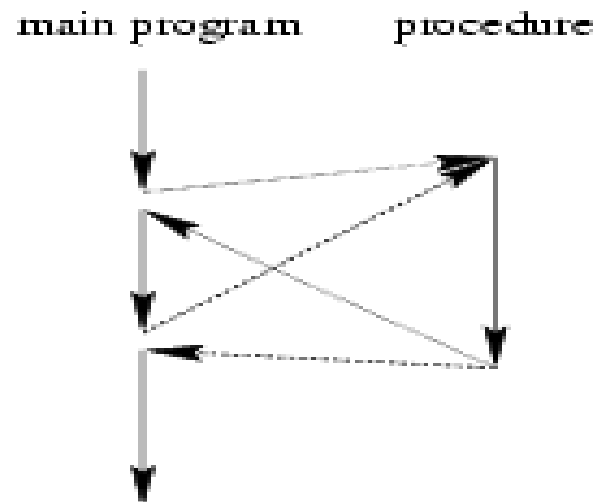
- consisting only of one large (usually main) program
- “main program” stands for a sequence of commands or *statements*
 - data is *global* throughout the whole program



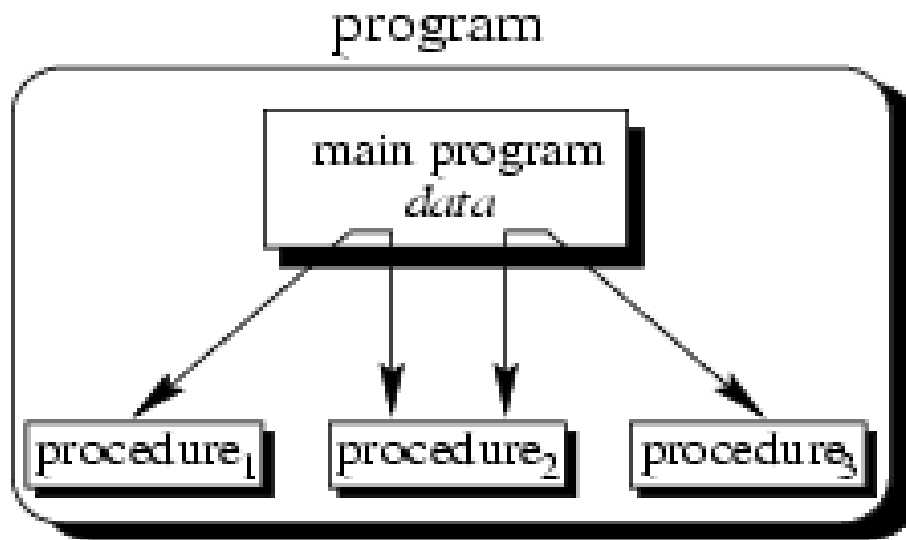
- disadvantages
 - Complex
 - if the same statement sequence is needed at different locations within the program, the sequence must be copied

Procedural programming

- based upon the concept of *procedure call*
- A *procedure call* is used to invoke the procedure
- Procedures (routines, subroutines, methods, functions) simply contain a series of computational steps to be carried out to solve a problem



- We have a single program, which is divided into small pieces called procedures

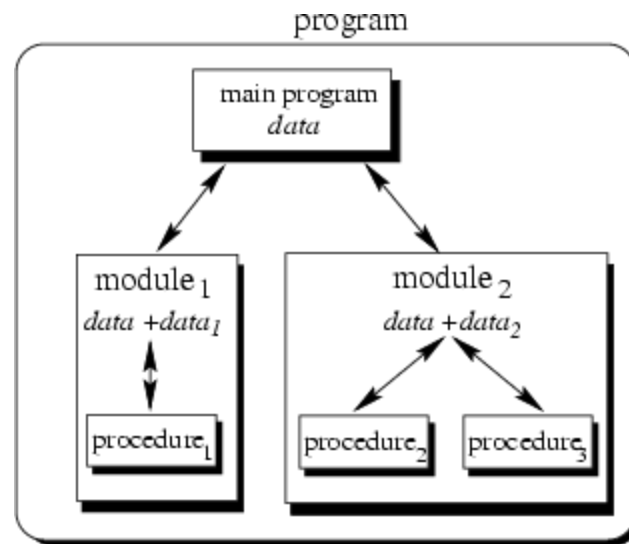


- Advantage
 - to re-use the same code at different places in the program without copying it
 - easier way to keep track of program flow
- Example
 - FORTRAN, ADA

Structured Programming

- is a subset of procedural programming (also known as *modular programming*)
- procedures of a common functionality are grouped together into separate *modules*
- Each module can have its own data
 - allows each module to manage an internal *state* which is modified by calls to procedures of this module
- top-down design model
 - map out the overall program structure into separate subsections
- Example
 - PASCAL, C

- Advantage
 - Reuse
 - easier to understand and modify



Object Oriented Programming - OOP

- Is a method of implementation in which programs are organized as cooperative collections of objects
- Data and operations are grouped together
- Each object is capable of receiving messages, processing data, and sending messages to other objects
- Modeling of the domain as objects so that the implementation naturally reflects the problem at hand.

Problem solving Techniques

- Computer solves varieties of problems that can be expressed in a finite number of steps
- A program
 - solve a problem
 - should be reliable, (maintainable) portable and efficient
- In computer programming two facts :
 - Defining the problem and logical procedures to follow in solving it.
 - Introducing the means by which programmers communicate those procedures to the computer system so that it can be executed.

Programs vs. Algorithms

- To make a computer do anything, you have to write a program
- To write a computer program, you have to tell the computer, step by step, exactly what you want it to do.
- The computer then "executes" the program, following each step mechanically, to accomplish the end goal.
- An *algorithm* is a finite, step-by-step sequence of instructions that describe how the data is to be processed to produce **the desired outputs**
- The algorithm is implemented by a program

Control Structures

- programs could be written in terms of 3 structures:
 - Sequence – which instruction should be done next?
 - Selection – select between options
 - Repetition – repeat an action while a given condition stays true

- An algorithm can be represented as
 - Pseudocode
 - Flowchart
 - Structured chart

Pseudocode

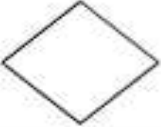
- is an artificial and informal language that helps programmers develop algorithms
- allows the designer to focus on the logic of the algorithm without being distracted by details of language syntax
- text-based" detail (algorithmic) design tool
- Pseudocode is not a rigorous notation, since it is read by other people, not by the computer

- Computation/Assignment
 - set the value of "variable" to : "arithmetic expression" or
"variable" equals "expression" or
"variable" = "expression"
- Input/Output
 - **get** "variable", "variable", ...
display "variable", "variable", ...
- Conditional
 - **if** "condition"
 (subordinate) statement 1
 etc ...
else
 (subordinate) statement 2
 etc ...
- Iterative
 - **while** "condition"
 (subordinate) statement 1
 etc ...

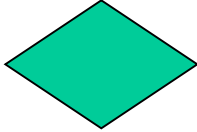
- Examples:
 - Write a program that prints “passed” when the student grade is greater than 60 and “failed” otherwise.
 - If student's grade is greater than or equal to 60
 - Print "passed"
 - else
 - Print "failed"
 - Pseudo-code the task of computing the final price of an item after figuring in sales tax. Note the three types of instructions: input (**get**), process/calculate (**=**) and output (**display**)
 1. **get** price of item
 2. **get** sales tax rate
 3. sales tax **=** price of item times sales tax rate
 4. final price **=** price of item plus sales tax
 5. **display** final price
 6. **stop**
- Variables: price of item, sales tax rate, sales tax, final price

Flow Chart

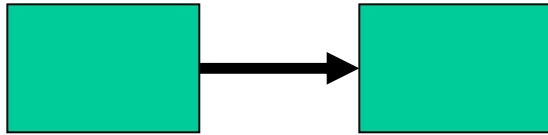
- A graphic representation of an algorithm, often used in the design phase of programming to work out the logical flow of a program

Symbol	Name	Function
	Terminal	Used to represent the start of the end of a program
	Input/ output	Used to represent data input or data output from a computer
	Processing	Usually encloses operations or (command block) a group of operations(a process)
	Decision block	it usually contains a question within it there are typically two output paths: one if the answer to the question is yes (true) , and the other if the answer is no (false)
	Flow line	is used to indicate the direction of logical flow (a path from one operation to another
	On-page	is used for connecting two points in connector a flow chart without drawing flow lines In one page.
	Off page connector	It is used an exit to or any entry from another part of the flowchart on another page

Continued...

- Action Symbol 
 - Instructions changing a state
- Decision Symbol 
 - Instructions testing a state
- Flowline 
 - Instructions transferring to next step
- Start/End Symbol 

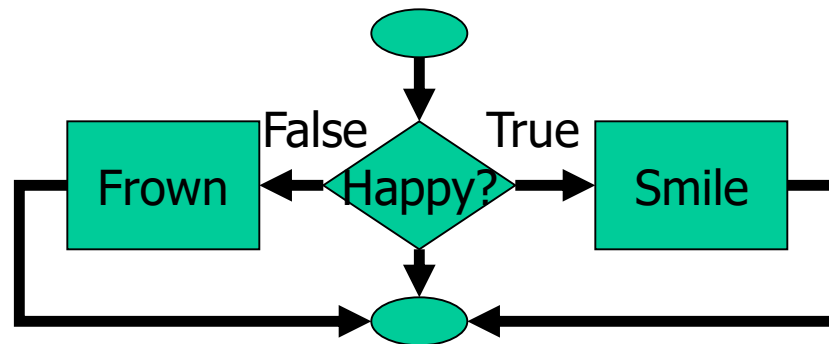
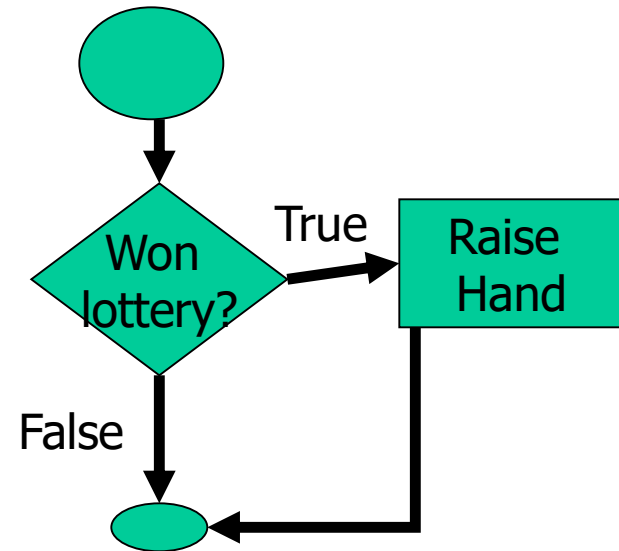
Sequence



- 1) Open the jar
- 2) Scoop out the peanut butter
- 3) Spread the peanut butter on the bread

Selection

- Single Selection (if)
 - If you've won the lottery:
raise your hand
- Double Selection (if-else)
 - If you're happy:
smile
 - else:
frown



Selection (continued)

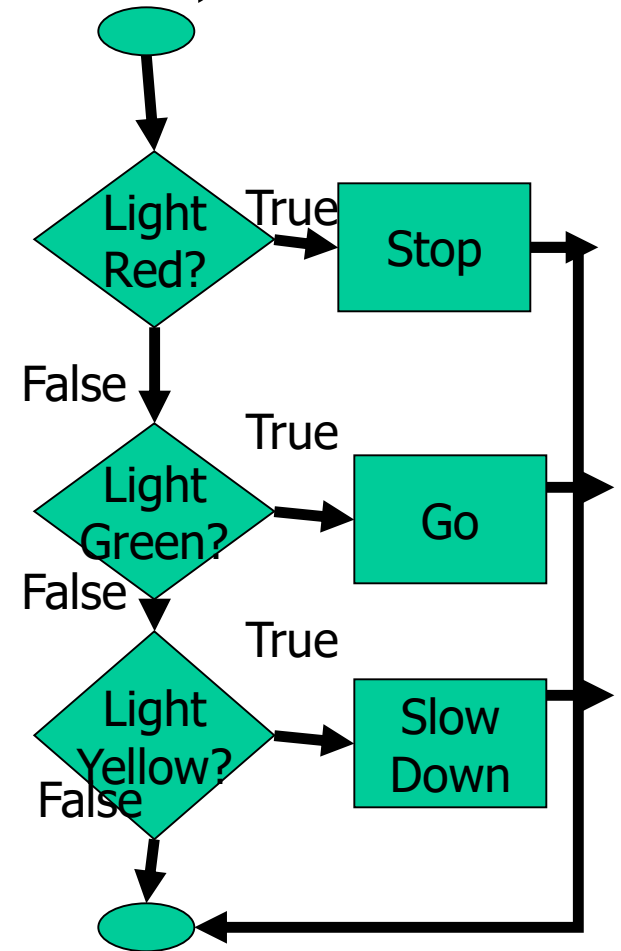
- Multiple Selection (switch)

If the light is ...

red -> stop

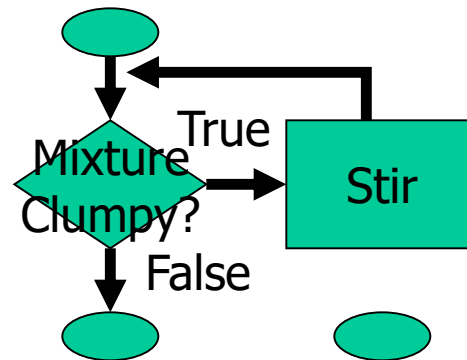
green -> go

yellow -> slow down



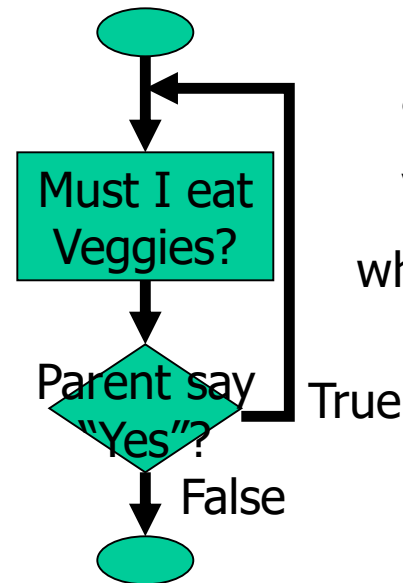
Repetition

- While



while it's still clumpy
Stir the mixture

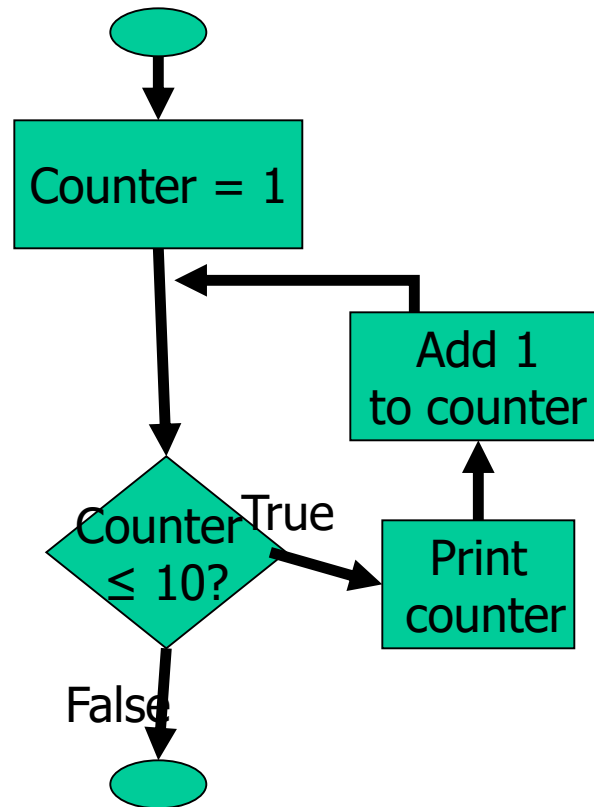
- Do-while



ask parents if must eat
vegetables
while parents say "Yes"

Repetition (continued)

- For



Teaching a baby to count
from 1 to 10:

counter = 1

if counter ≤ 10:

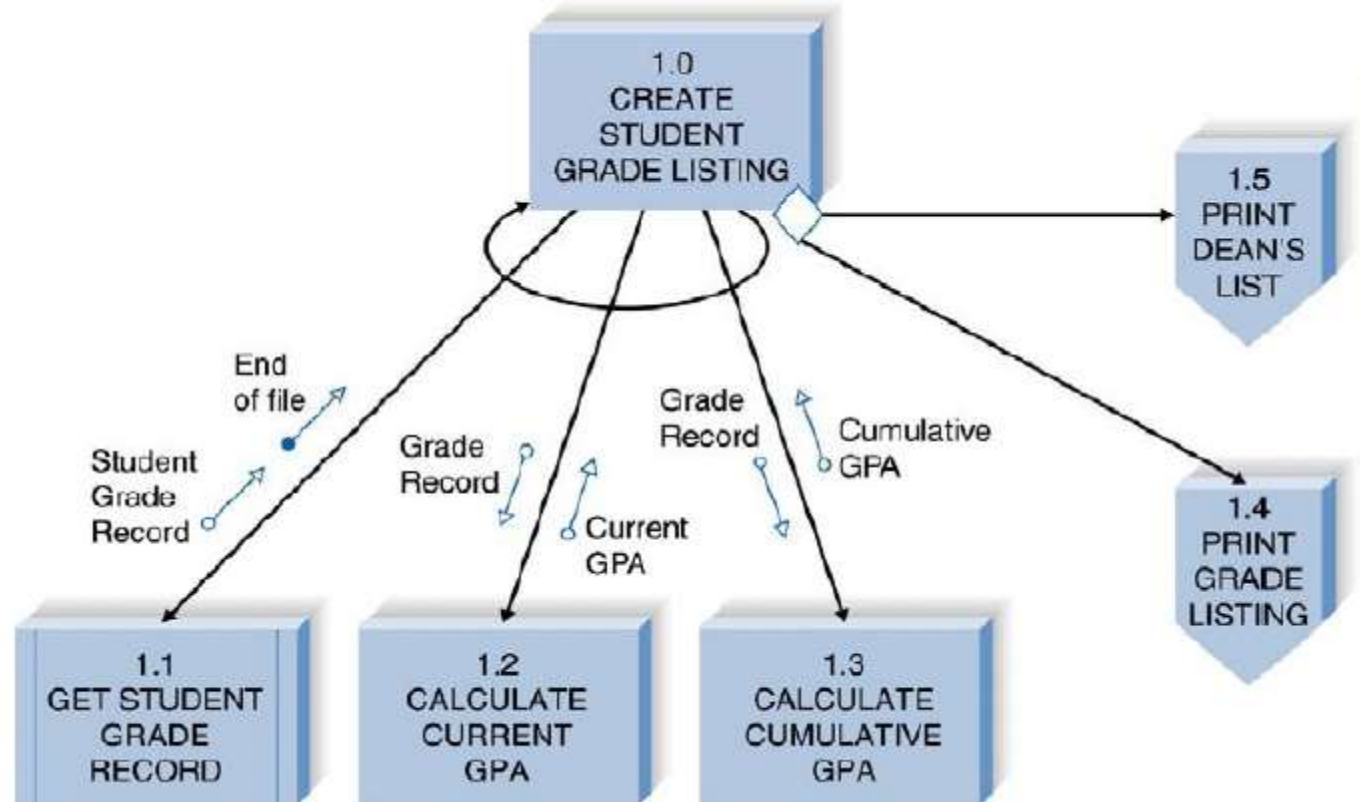
 increment counter

 print counter number

Structure Chart

- The structure chart is an important technique that helps the analyst design the program for the new system.
- There are three components in the structured chart: modules, connections between modules, and communication between modules.

Structure Chart Example

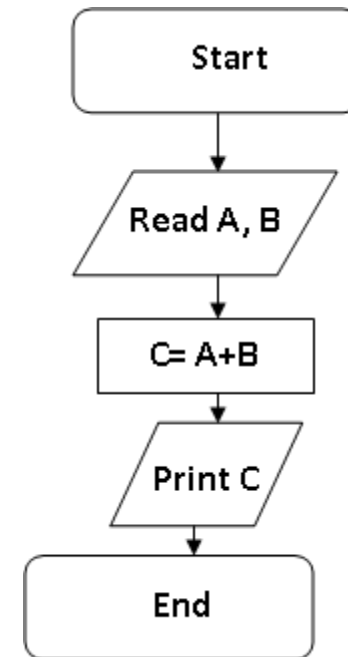


Problems

- Example 1: - Draw flow chart of an algorithm to add two numbers and display their result.

Algorithm description

- Read the values of the two numbers (A and B)
- Add A and B
- Assign the sum of A and B to C
- Display the result (c)



The flow Chart

Continued.....

- Example 2: Write an algorithm description and draw a flow chart to check a number is negative or not.
- Example 3: - Write the algorithmic description and draw a flow chart to find the following sum.
 - $\text{Sum} = 1+2+3+\dots + 50$